

山东大学



网络安全创新创业实践课程作业二

secp256k1 仓库缺陷修复与性能优化分析

成员姓名	学号	班级	主要分工
马悦曦	202300201057	网安二班	报告撰写、secp256k1 仓库分析与 ECDSA 伪造攻击数学推导
周思琪	202300450129	密码一班	签名与多梳算法性能优化分析
闫雨彤	202322161173	密码一班	ECDH 计时侧信道与 nonce 归约缺陷分析
孙世瑾	202300210010	密码一班	域运算性能优化与 LaTeX 排版整理

2026 年 7 月 25 日

目录

1 TASK 2 : ECDSA — 当签名消息未被检查时伪造签名	1
1.1 ECDSA 基本回顾	1
1.2 消息未被检查时的伪造攻击	1
2 secp256k1 仓库中的缺陷修复与性能优化	2
2.1 修复一: ECDSA RFC 6979 nonce 生成中的消息摘要归约	2
2.1.1 问题描述	2
2.1.2 修复提交	2
2.1.3 数学原因	2
2.2 修复二: ECDH 的常数时间/计时侧信道修复	3
2.2.1 ECDH 与常数时间	3
2.2.2 Clang ≥ 14 <i>cmov</i>	3
2.2.3 ECDH 偏移修正的性能优化	3
2.3 性能优化一: 签名/公钥生成的 signed-digit multi-comb 算法	4
2.3.1 背景与提交	4
2.3.2 算法数学	4
2.3.3 性能收益与配置	4
2.4 性能优化二: 域运算的 force-inline 与序列化重构	4
2.4.1 5x52 域表示	4
2.4.2 Force-inline 乘法与平方	5
2.4.3 序列化按字打包	5
3 数学原理总结	5
4 结论	6

1 TASK 2 : ECDSA — 当签名消息未被检查时伪造签名

1.1 ECDSA 基本回顾

secp256k1 曲线方程为 $y^2 = x^3 + 7 \pmod{p}$ ，其基点为 G ，群的阶为素数 n 。私钥为 $d \in \mathbb{Z}_n^*$ ，公钥为 $P = dG$ 。对消息 m 的 ECDSA 签名流程如下：

1. 随机选择 nonce $k \leftarrow \mathbb{Z}_n^*$ ，计算 $R = kG = (R_x, R_y)$ ；
2. 令 $r = R_x \bmod n$ ，若 $r = 0$ 则重新选择 k ；
3. 计算消息摘要 $e = \text{hash}(m)$ ，并视作一个整数；
4. 计算 $s = k^{-1}(e + dr) \bmod n$ ，若 $s = 0$ 则重新选择 k ；
5. 签名为 $\sigma = (r, s)$ 。

验证者使用公钥 P 检查 (r, s) 是否有效：

1. 计算 $e = \text{hash}(m)$ ， $w = s^{-1} \bmod n$ ；
2. 计算 $R' = ewG + rwP$ ；
3. 令 $r' = R'_x \bmod n$ ，检查 $r' \stackrel{?}{=} r$ 。

上述验证等式成立，因为

$$s^{-1}(eG + rP) = k(e + dr)^{-1}(e + dr)G = kG = R.$$

1.2 消息未被检查时的伪造攻击

在某些接口或协议中，验证者只要求提供一个“哈希值” e 而非原始消息 m 。如果攻击者能够任意指定这个 e ，那么即使不知道私钥 d ，也可以构造出对任意公钥 P 有效的签名。具体方法如下：

攻击者任选 $u, v \in \mathbb{Z}_n^*$ ，计算

$$R' = (x', y') = uG + vP.$$

令 $r' = x' \bmod n$ 。为了使 (r', s') 通过验证，需要满足

$$s'^{-1}(e'G + r'P) = uG + vP.$$

这等价于

$$s'^{-1}e' = u \bmod n, \quad s'^{-1}r' = v \bmod n.$$

由此解出

$$s' = r'v^{-1} \bmod n, \quad e' = r'uv^{-1} \bmod n.$$

于是 (r', s') 就是对“消息摘要” e' 的一个合法签名，而攻击者完全不知道 d 。该攻击的核心在于：验证等式只涉及 $eG + rP$ 的线性组合，因此任何能表示成 $uG + vP$ 形式的点都可以被包装成合法签名。

防范措施：验证时必须对原始消息 m 进行哈希（必要时先检查 m 的语义），并拒绝由外部直接传入的未经验证的 e 。在 Bitcoin 中，签名验证通常与交易摘要绑定，因此该攻击在标准协议路径下不直接生效；但在构建库或自定义协议时，必须牢记这一边界条件。

2 secp256k1 仓库中的缺陷修复与性能优化

本报告对本地仓库 `/home/yyt/Desktop/secp256k1` 及上游 <https://github.com/bitcoin-core/secp256k1> 中与密码学算法使用相关的修复和优化进行了梳理。下面分别介绍两项安全/正确性修复和两项性能优化，并给出背后的数学原理。

2.1 修复一：ECDSA RFC 6979 nonce 生成中的消息摘要归约

2.1.1 问题描述

在 `libsecp256k1 v0.2.0` 之前的版本中，`secp256k1_nonce_function_rfc6979` 在生成确定性 nonce 时，将调用者传入的 32 字节消息摘要 `msg32` 直接作为 HMAC 种子的一部分，而未按 RFC 6979 的要求先将其对群阶 n 取模。RFC 6979 第 2.3.4 与 3.2d 节明确规定：

$$\text{bits2octets}(h_1) = \text{int2octets}(\text{bits2int}(h_1) \bmod n).$$

2.1.2 修复提交

相关提交为 `45f37b6`，由 Paul Miller 于 2022-01-17 提交，合并提交为 `c8aa516`。修改仅涉及文件 `src/secp256k1.c` 中的 `nonce_function_rfc6979` 函数。修复代码如下：

Listing 1: RFC 6979 nonce 生成中的消息摘要归约修复

```
1 secp256k1_scalar msg;  
2 unsigned char msgmod32[32];  
3 secp256k1_scalar_set_b32(&msg, msg32, NULL); /* 模 n 归约 */  
4 secp256k1_scalar_get_b32(msgmod32, &msg); /* 序列化回 32 字节 */  
5 ...  
6 /* 后续将 msgmod32 而非 msg32 填入 HMAC 种子 */
```

2.1.3 数学原因

secp256k1 的群阶

$$n = 2^{256} - \Delta,$$

其中

$$\Delta = 0x14551231950B75FC4402DA1732FC9BEBF$$

略小于 2^{256} 。因此，一个随机的 256 位哈希值大于 n 的概率约为 2^{-32} ，虽不频繁但不可忽略。ECDSA 的签名方程本身会把 e 对 n 取模，因此两个在模 2^{256} 下不同、但在模 n 下相等的摘要值对验证者而言是同一个消息。RFC 6979 要求它们产生同一个 nonce，但旧实现会因高位的差异而生成不同的 nonce，从而破坏签名的唯一性。

更严重的是，如果攻击者能够控制传入 HMAC 的种子高位，就能影响 nonce 的分布；而 nonce 的偏差正是多种格攻击（如 Bleichenbacher 类攻击）恢复私钥的前提。因此，该修复是一个与伪造能力相关的标准合规性修正。正确用法（即调用者已经传入模 n 归约后的摘要）不受影响。

2.2 修复二：ECDH 的常数时间/计时侧信道修复

2.2.1 ECDH 与常数时间

ECDH 模块 (`src/modules/ecdh/main_impl.h`) 计算共享密钥

$$S = dP,$$

其中 d 是私钥, P 是对方公钥。该计算委托给 `secp256k1_ecmult_const(src/ecmult_const_impl.h)`, 其必须对 d 的每一位都保持常数时间: 访问的内存地址和执行路径都不能依赖秘密标量。

2.2.2 Clang ≥ 14 *cmov*

2019 年之后, 编译器对 `cmov` 式常数时间选择函数进行激进优化。`libsecp256k1` 的 `secp256k1_fe_cmov`、`secp256k1_fe_storage_cmov` 和 `secp256k1_scalar_cmov` 原本通过位掩码实现:

Listing 2: `libsecp256k1` 的 `cmov` 位掩码实现

```
1 mask0 = vflag + ~((uint64_t)0);
2 mask1 = ~mask0;
3 r->n[0] = (r->n[0] & mask0) | (a->n[0] & mask1);
```

Clang 14+ 可能将上述位运算优化为带分支的

Listing 3: 编译器优化后可能产生的分支代码

```
1 if (flag) *r = *a;
```

从而产生秘密依赖的控制流。修复提交 4a496a3 (PR #1257, v0.3.1) 在条件变量上加了 `volatile`:

Listing 4: 使用 `volatile` 阻止编译器优化分支

```
1 volatile int vflag = flag;
```

`volatile` 阻止编译器在编译期知道 `vflag` 的值, 从而无法将其转换为分支。同样的方法后来被应用到 `secp256k1_scalar_cadd_bit`、`secp256k1_scalar_cond_negate` 以及模逆函数 `modinv32/modinv64` 中 (PR #1303, v0.3.2, GCC 13 问题)。

2.2.3 ECDH 偏移修正的性能优化

更早的提交 1515099 (PR #1029, v0.2.0) 简化了 WNAF 阶梯中的 `scalar` 偏移修正。原算法为了保证标量奇数化, 向标量加了 1 或 2 的偏移, 最后需要减去相应倍数的基点; 旧实现通过一次昂贵的 Jacobian 到仿射坐标转换 (含域逆) 来计算修正点。新实现改为:

Listing 5: ECDH 偏移修正的常数时间选择实现

```
1 secp256k1_gej_add_ge(r, r, &a_1); /* 总是先减一个 a */
2 secp256k1_gej_add_ge(&tmp, r, &a_1); /* tmp = 再减一个 a */
3 secp256k1_gej_cmov(r, &tmp, skew_1); /* 条件选择 */
```

其中 `secp256k1_gej_cmov` 使用 `secp256k1_fe_cmov` 对三个坐标做常数时间选择。数学上, 若阶梯计算的是 $R = [(q + \text{skew})]A$, 则最终需要返回 $qA = R - \text{skew} \cdot A$ 。新实现通过预先计算 $-A$ 并在常数时间内选择是否再减去一个 $-A$, 既避免了域逆, 又把偏移范围从 $\{1, 2\}$ 缩小到 $\{0, 1\}$, 使得 ECDH 基准测试提速约 5% (i7 64 位)。

2.3 性能优化一：签名/公钥生成的 signed-digit multi-comb 算法

2.3.1 背景与提交

对固定基点 G 的标量乘法 `ecmult_gen` 在签名和公钥生成中处于核心路径。2021 年，Peter Dettman 提出并实现了一种 signed-digit multi-comb 算法 (提交 `fde1dfc`, PR #1058, v0.5.0), 其理论基础来自 Mike Hamburg 的论文 *Fast and compact elliptic-curve cryptography* (ePrint 2012/309) 第 3.3 节。

2.3.2 算法数学

设标量 k 为 256 位。算法将 k 分成 B 个块 (blocks), 每块有 T 个齿 (teeth), 齿间距为 S 。对每个块 b , 定义掩码

$$\text{mask}(b) = \sum_{t=0}^{T-1} 2^{(bT+t)S}.$$

对任意可能的局部值 $m = d \wedge \text{mask}(b)$, 预计算

$$\text{table}(b, m) = \left(m - \frac{\text{mask}(b)}{2} \right) G.$$

利用对称性

$$\text{table}(b, \text{mask}(b) - m) = -\text{table}(b, m),$$

表大小可减半。在线计算为

$$\text{comb}(d, G/2) = \sum_{o=0}^{S-1} 2^o \sum_{b=0}^{B-1} \text{table}(b, (d \gg o) \wedge \text{mask}(b)).$$

为防止侧信道, 表中取点使用常数时间的 `secp256k1_ge_storage_cmov` 遍历整个表, 符号位通过 `secp256k1_fe_cmov` 对 y 坐标做条件取反。随机 blinding 通过 $R = (k - b)G + bG$ 实现, 其中 b 为随机标量。

2.3.3 性能收益与配置

该算法将默认预计算表大小从旧算法的 64 KiB 降至 22 KiB (v0.5.0 默认), 并在后续 v0.5.1 中改为 86 KiB 以进一步提速。PR #1058 中的测试显示:

- AMD Ryzen 5950X (GCC 11.2): `schnorr_sig_sign` 从 $18.1 \mu\text{s}$ 降至 $15.1 \mu\text{s}$ (22 KiB), 再降至 $14.4 \mu\text{s}$ (86 KiB);
- Intel i7-7820HQ (GCC 10.3): 从 $38.9 \mu\text{s}$ 降至 $33.3 \mu\text{s}$;
- ARM Cortex-A53 (GCC 9.3): 从 $249 \mu\text{s}$ 降至 $200 \mu\text{s}$ 。

合并后的独立测试在 GCC 13.2 下获得约 12.4% 的签名提速, Clang 15 下约 11.5%。

2.4 性能优化二：域运算的 force-inline 与序列化重构

2.4.1 5x52 域表示

`secp256k1` 的素域为

$$\mathbb{F}_p, \quad p = 2^{256} - 2^{32} - 977.$$

在 64 位平台上，域元素用 5×52 表示：

$$a = a_0 + a_1 2^{52} + a_2 2^{104} + a_3 2^{156} + a_4 2^{208} \pmod{p},$$

每个 a_i 为 52 位 (a_4 为 48 位)。这样两个 52 位数的乘积刚好落在 104 位，可由 128 位累加器 `secp256k1_uint128` 容纳。模约化利用

$$2^{256} \equiv 2^{32} + 977 \pmod{p},$$

即约化常数 $R = 0x1000003D10$ 。

2.4.2 Force-inline 乘法与平方

提交 71fcd84 (PR #1859, 尚未进入已发布版本, 在 v0.7.1 之后) 注意到, `secp256k1_fe_impl_mul` 与 `secp256k1_fe_impl_sqr` 作为热路径上的薄包装, 仅被标记为普通 `SECP256K1_INLINE`, GCC 与 MSVC 在某些 Release 构建中仍将其编译为真实函数调用, 阻碍了寄存器分配与 $64 \times 64 \rightarrow 128$ 乘法链的调度。

修复引入 `SECP256K1_FORCE_INLINE`: 在 GCC/Clang 上为 `always_inline`, 在 MSVC 上为 `__forceinline`, 在调试或大小优化构建中退化为普通 `inline`。实测性能提升:

- ECDSA 验证: 0.6% 至 9.1%;
- ECDH: 0.7% 至 9.3%;
- Schnorr 验证: 0.6% 至 9.6%;
- 域平方/乘法最高分别提升 15.7% 和 21.6%。

代价是静态库体积增加约 4.6% 至 4.7%, Windows Release DLL 增加约 14.1%。

2.4.3 序列化按字打包

提交 e217ead (PR #1889, v0.7.1 之后) 将 `field_5x52` 的序列化从 32 次逐字节赋值改为 4 个 64 位大端字写入, `field_10x26` 改为 8 个 32 位大端字写入。这使用了已有的 `secp256k1_write_be64/secp256k1_write_be32` 辅助函数, 与标量序列化风格保持一致。该改动未引入可测性能回退, 主要提高了可审计性。

3 数学原理总结

1. **ECDSA 伪造**: 验证方程 $R' = ewG + rwP$ 对 e 和 r 是线性的。只要攻击者能任意选择 e , 就可以把任意 $uG + vP$ 重新参数化为合法签名 (r', s') , 这是协议设计中“必须检查消息来源”的根本原因。
2. **nonce 归约**: 群阶 n 略小于 2^{256} 。ECDSA 与 RFC 6979 的语义都以模 n 的标量为操作对象, 因此未归约的 256 位摘要会破坏确定性映射的唯一性与安全性论证。
3. **常数时间**: 编译器优化可能将位掩码选择转化为分支。`volatile` 条件变量是一种实用的编译器屏障, 可阻止这种转换, 但需在每个可能泄露秘密比特的函数中持续维护。

4. **comb 标量乘法**: 对固定基点的乘法可通过预计算与 signed-digit 对称性减少在线运算量。常数时间实现要求表查找与符号选择都通过 `cmov` 完成, 而非数组索引或分支。
5. **域运算**: `5x52` 表示把 256 位素域拆成 5 个 52 位肢, 使 $64 \times 64 \rightarrow 128$ 乘法链能被现代编译器高效调度。强制内联消除了调用边界, 是纯粹的工程优化。

4 结论

secp256k1 作为 Bitcoin 核心的密码学库, 其演进体现了密码学实现中安全性、标准合规性与性能之间的持续权衡。本报告分析了与 Bitcoin 使用场景密切相关的两类修复:

- **安全/正确性修复**: RFC 6979 nonce 的摘要归约修复, 以及 Clang/GCC 常数时间优化导致的 ECDH 计时侧信道修复。两者都说明, 数学上正确的算法并不能自动保证实现安全, 编译器优化与接口误用都会引入风险。
- **性能优化**: signed-digit multi-comb 算法将签名/公钥生成路径的预计算表更小、更快; force-inline 的域乘法/平方则通过消除编译器调用边界, 进一步提升了 ECDSA/Schnorr 验证与 ECDH 的吞吐量。

这些改动无一例外都根植于椭圆曲线标量乘法、素域运算和常数时间编程的数学基础。对 Bitcoin 而言, 更快的签名验证意味着更高的节点吞吐能力; 更严密的常数时间保证则保护着无数用户的私钥免受侧信道攻击; 而 nonce 生成与消息摘要的正确处理, 是防止签名伪造与私钥泄露的第一道防线。

参考文献

- [1] bitcoin-core/secp256k1, *secp256k1*, <https://github.com/bitcoin-core/secp256k1>.
- [2] T. Pornin, *Deterministic Usage of DSA and ECDSA*, RFC 6979, 2013.
- [3] M. Hamburg, *Fast and compact elliptic-curve cryptography*, ePrint 2012/309.
- [4] bitcoin-core/secp256k1, *CHANGELOG.md*, <https://github.com/bitcoin-core/secp256k1/blob/master/CHANGELOG.md>.
- [5] bitcoin-core/secp256k1, PR #1058, *Signed-digit multi-comb ecmult_gen algorithm*, <https://github.com/bitcoin-core/secp256k1/pull/1058>.
- [6] bitcoin-core/secp256k1, PR #1257, *Fix cmov constant-time issues with Clang 14+*, <https://github.com/bitcoin-core/secp256k1/pull/1257>.
- [7] bitcoin-core/secp256k1, PR #1303, *Fix GCC 13 ECDH constant-time issue*, <https://github.com/bitcoin-core/secp256k1/pull/1303>.
- [8] bitcoin-core/secp256k1, PR #1859, *field: force-inline 5x52 mul and sqr*, <https://github.com/bitcoin-core/secp256k1/pull/1859>.